



PicoScript

Compiler Construction Project

Digital Documentation

Submitted by:

Syeda Batool Kazmi, 23K-0672

Rabia Zulfiqar, 23K-0851

Submitted to:

Sir Muhammad Kashif

Submission date:

10th May 2026

B.4 Language Reference Manual

PicoScript is a domain-specific language (DSL) designed for authoring interactive text-based adventure games in the spirit of 8-bit RPGs. It provides a declarative, human-readable syntax for defining game worlds, rooms, items, puzzles, and NPCs without requiring deep programming knowledge.

Design Principles

- **Declarative:** Define game elements as self-contained blocks
- **Puzzle-Centric:** Native support for puzzle gates and key-based access systems
- **Player-Driven:** Focus on player actions and interactions
- **Type-Safe:** Flags are boolean and immutable; changed only by puzzle gates

File Format

PicoScript programs use the .pico file extension and are executed via:

```
python3 main.py game.pico
```

Lexical Elements

Keywords

Reserved words that cannot be used as identifiers:

world	room	item	flag	player	inv
go	say	win	lose	requires	has
unlock	puzzle	gate	on	enter	examine
use	talk	give	take	drop	npc
if	else	end	true	false	
north	south	east	west		

Identifiers

Begin with a letter or underscore, followed by alphanumeric characters or underscores. Case-sensitive.

```
valid:    my_room, key, entrance_2, _private
invalid:  123abc, my-room, my room
```

String Literals

Enclosed in double quotes; printed as-is in the game.

```
"An old iron key."
"You escaped the crypt!"
```

Operators & Punctuation

Token	Meaning
->	Arrow – used for exit declarations
:	Colon – block opener
=	Equals – flag assignment
//	Comment – rest of line is ignored

Program Structure

Every PicoScript program consists of a single world block:

```
world <name>:
    <declarations>
end
```

The world body can contain (in any order): flag declarations, a player block, item declarations, room declarations, puzzle declarations, and NPC declarations.

Minimal Example

```
world Crypt:
  flag door_open = false
  player:
    inv key
  end
  item key:
    examine: "An old iron key."
  end
  room entrance:
    item key
    go north -> chamber requires flag door_open
  end
  room chamber:
    on enter:
      win "You escaped!"
    end
  end
  puzzle gate:
    requires inv has key
    unlock door_open
  end
end
```

Declarations

Flags

Global boolean variables representing game state.

```
flag <name> = <true|false>
```

Semantics:

- Initialized at game start
- Immutable in code – changed only by puzzle gates during execution
- Globally scoped and must be unique per world
- Can be checked in conditions

Player Block

Defines the player's starting inventory. Optional; if omitted, the player starts with an empty inventory.

```
player:
  inv <item_name>
  ...
end
```

Items

Interactive objects that players can examine and use. Items are globally declared and can be placed in multiple rooms.

```
item <name>:
  examine: "<description>"
  use:
    <statements>
  end
end
```

Rooms

Locations in the game world that players navigate between. The first declared room is the starting room.

```
room <name>:
  item <item_ref>
  go <direction> -> <target_room>
  go <direction> -> <target_room> requires <condition>
  on enter:
    <statements>
  end
end
```

Valid directions:

- north, south, east, west

Puzzles

Conditions that unlock flags when satisfied. Puzzle gates are checked continuously during gameplay; when the condition becomes true the flag is automatically set.

```
puzzle gate:
  [room <room_name>]
  requires <condition>
  unlock <flag_name>
end
```

NPCs

Non-player characters with dialogue. Dialogue blocks may include conditional if/else/end branches.

```
npc <name>:
  [room <room_name>]
  talk:
    [if <condition>
      <statements>
    else
      <statements>
    end]
  end
end
```

Conditions

Boolean expressions used in puzzle requirements, exit requirements, and NPC dialogue.

Syntax	Evaluates to true when...
flag <flag_name>	The named flag is currently true
inv has <item_name>	The player has the item in their inventory

Statements

Executable actions within item use blocks, on-enter blocks, and NPC dialogue.

Statement	Syntax	Effect
say	say "<text>"	Print a message to output
win	win "<text>"	Print message, halt execution, mark game won
lose	lose "<text>"	Print message, halt execution, mark game lost

Runtime Commands

During gameplay the player interacts using the following text commands:

Command	Syntax	Effect
Look	look	Display current room description
Inventory	inv	List items in inventory
Go	go <direction>	Move to adjacent room
Take	take <item>	Pick up item from room
Drop	drop <item>	Drop item into room
Examine	examine <item>	View item description
Use	use <item>	Execute item's use action
Talk	talk <npc>	Trigger NPC dialogue
Help	help	Show available commands
Quit	quit / exit	Exit the game

Semantic Constraints

1. Unique Names – All flags, items, rooms, and NPCs must have unique names
2. Declaration Before Use – Items must be declared before placement; rooms before reference
3. At Least One Room – A world must contain at least one room
4. Valid Conditions – All flags and items in conditions must be declared
5. No Circular Dependencies – Forward references allowed but must resolve

Common Error Messages

Error	Cause
unexpected token in world body	Invalid declaration at world scope
expected true or false	Flag not initialized with boolean literal
undefined room	Exit references a non-existent room
undefined item	Item referenced but not declared
duplicate <element> declaration	Name already used for the same element type
player inventory references undeclared item	Item in inv block was not declared

Example Programs

Example 1 – Simple Key & Door

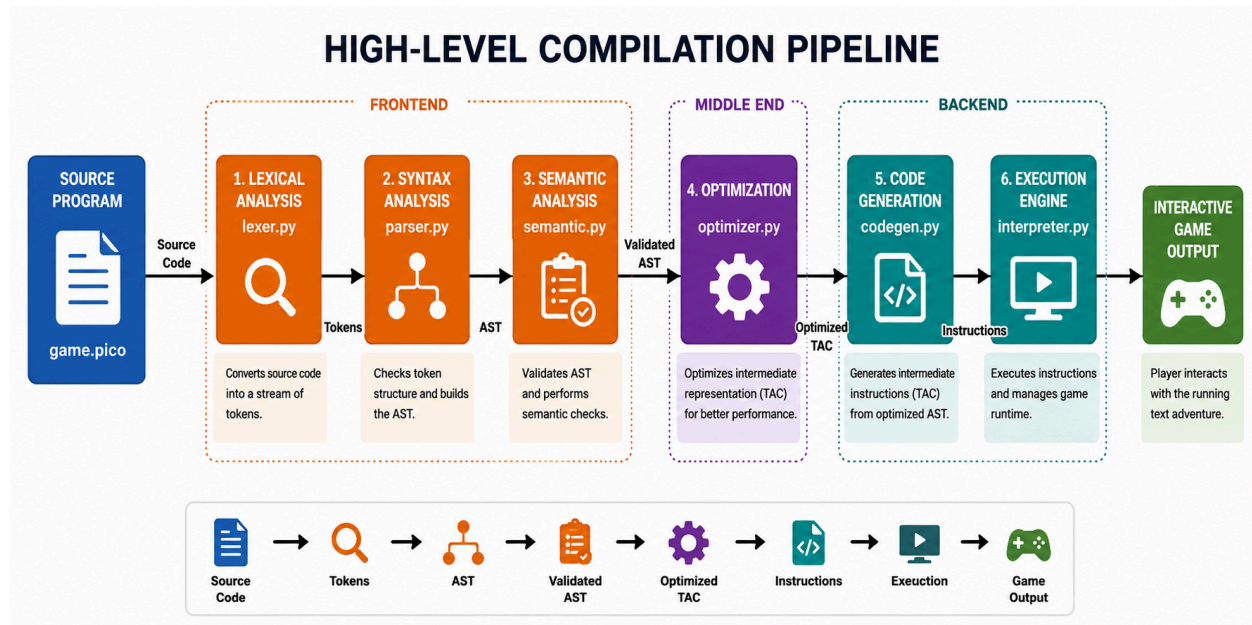
```
world SimpleKey:
  flag door_open = false
  player:
    inv key
  end
  item key:
    examine: "A rusty key."
  end
  room entrance:
    go north -> treasure requires flag door_open
  end
  room treasure:
    on enter:
      win "You found the treasure!"
    end
  end
  puzzle gate:
    requires inv has key
    unlock door_open
  end
end
```

Example 2 – NPC with Conditional Dialogue

```
world NPCDialogue:
  flag helped = false
  player:
  end
  npc sage:
    talk:
      if flag helped
        say "Thank you for your help!"
      else
        say "Please help me escape."
      end
    end
  end
  room chamber:
  end
end
```

B.5 Compiler Architecture Document

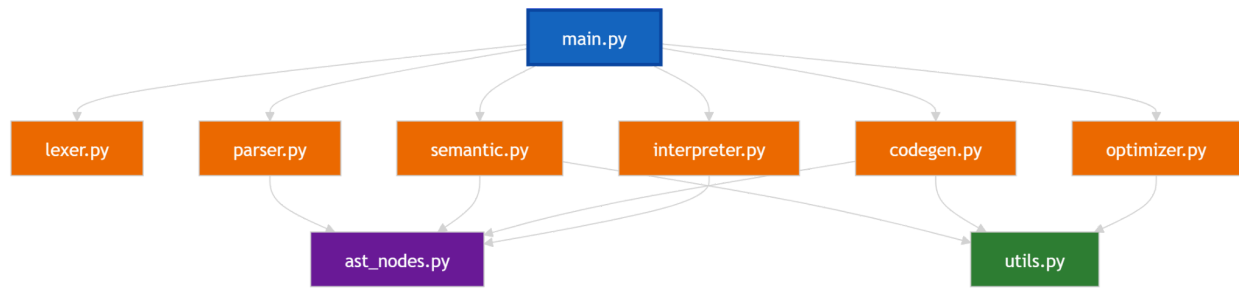
The PicoScript compiler is a multi-phase interpreter that transforms high-level game specifications into executable text adventures. The architecture follows a traditional compiler pipeline with six distinct phases.



Compiler Pipeline Overview

Phase	Module	Input	Output
1 – Lexical Analysis	lexer.py	Raw source text	Token stream
2 – Syntax Analysis	parser.py	Token stream	Abstract Syntax Tree (AST)
3 – Semantic Analysis	semantic.py	AST	Validated AST + Symbol Table
4 – Code Generation	codegen.py	AST	Three-Address Code (TAC)
5 – Optimization	optimizer.py	TAC	Optimized TAC
6 – Execution	interpreter.py	Optimized TAC + AST	Interactive game

Module Dependency Graph



```
main.py
├── lexer.py      (tokenise source)
├── parser.py     → ast_nodes.py
├── semantic.py   → ast_nodes.py
├── codegen.py    → ast_nodes.py
├── optimizer.py
└── interpreter.py → ast_nodes.py
```

Phase 1 – Lexical Analysis (lexer.py)

The lexer scans source text character-by-character using a master regex pattern that matches tokens in priority order: comments are skipped, strings are captured with quotes stripped, operators are classified, whitespace is skipped while line numbers are tracked, and bare words are classified as keywords or identifiers.

Token Patterns (priority order)

```
COMMENT  →  //[^\n]*           # Skipped
STRING   →  "[^"]*"           # Captured; quotes removed
ARROW    →  ->
COLON    →  :
EQUALS   →  =
NEWLINE  →  \n                # Line counter
SKIP     →  [ \t]+            # Discarded
WORD     →  [a-zA-Z_][a-zA-Z0-9_]*
```

Token Structure

```
class Token:
    type: str    # e.g. FLAG, IDENT, STRING
    value: str   # e.g. 'door_open', 'A key.'
    line: int    # for error reporting
```

Example

Input: `flag door_open = false`

```
Token(FLAG,    'flag',      line=1)
Token(IDENT,   'door_open', line=1)
Token(EQUALS,  '=',         line=1)
Token(FALSE,   'false',     line=1)
```

Complexity: $O(n)$ where n = source length. Single-pass scan.

Phase 2 – Syntax Analysis (parser.py)

A recursive-descent parser implementing an LL(1) grammar. Each grammar production is a method; lookahead is one token (via peek/match). The parser produces an AST without backtracking in $O(t)$ time where t = token count.

Grammar (simplified EBNF)

```
program          → world_block EOF
world_block      → 'world' IDENT ':' world_body 'end'
world_body       → (flag_decl | player_block | item_block
                    | room_block | puzzle_block | npc_block)*
flag_decl        → 'flag' IDENT '=' ('true' | 'false')
player_block     → 'player' ':' ('inv' IDENT)* 'end'
item_block       → 'item' IDENT ':' (examine_action | use_action)* 'end'
room_block       → 'room' IDENT ':' room_body 'end'
exit             → 'go' direction '->' IDENT ['requires' condition]
puzzle_block     → 'puzzle' 'gate' ':' ['room' IDENT]
                  'requires' condition 'unlock' IDENT 'end'
condition        → ('flag' IDENT) | ('inv' 'has' IDENT)
statements       → (('say' | 'win' | 'lose') STRING)*
```

Key AST Node Types

Node	Purpose
WorldNode	Root: world name and body
FlagNode	Flag declaration
PlayerNode	Player starting inventory
ItemNode	Item with actions
ExamineAction	Item examination description
UseAction	Item use statements
RoomNode	Room definition
ExitNode	Room exit with optional condition
OnEnterBlock	Room entry trigger
PuzzleNode	Puzzle gate
NPCNode	NPC character
ConditionFlag	Flag-based condition
ConditionInv	Inventory-based condition

Phase 3 – Semantic Analysis (semantic.py)

The semantic analyser validates the AST for correctness and builds a symbol table used by later phases. It performs two passes over the AST.

Pass 1 – Declaration Collection

Records all globally declared flags, items, rooms, and NPCs. Checks for duplicate names. Builds per-room item scopes.

Pass 2 – Reference Resolution

Validates that all references (room exits, item placements, conditions) point to declared entities. Validates player inventory consistency.

Symbol Table Structure

```
symbols = {
  "world": {
    "flags":      { "door_open": "bool", ... },
    "items":      { "key", "gem" },
    "rooms":      { "entrance", "chamber" },
    "player_inv": { "key" },
  },
  "rooms": {
    "entrance": { "items": {"key"} },
    "chamber":  { "items": set() },
  },
}
```

Validation Rules

6. Uniqueness – flags, items, rooms, NPCs each have unique names
7. Reference validity – all room, item, and flag references must exist
8. Inventory consistency – player inv items must be declared
9. Condition validity – conditions reference declared entities
10. Minimum – at least one room must be declared

Phase 4 – Code Generation (codegen.py)

Lowers high-level game constructs into Three-Address Code (TAC), a low-level intermediate representation where each instruction has at most three operands. Complexity: $O(a)$ where a = AST node count.

TAC Instruction Set

Instruction	Meaning
t1 = inv_has(key)	Test if player has item
t2 = flag(door_open)	Read flag value
if t1 goto LABEL	Conditional branch
goto LABEL	Unconditional jump
PRINT "message"	Output text
HALT	Stop execution
LABEL:	Jump target

Puzzle Gate – TAC Example

Input AST: PuzzleNode(condition=ConditionInv('key'), unlock_flag='door_open')

```
t1 = inv_has(key)
if t1 goto PUZZLE_UNLOCKED_1
goto PUZZLE_END_1
PUZZLE_UNLOCKED_1:
door_open = true
PUZZLE_END_1:
```

Room On-Enter – TAC Example

Input AST: OnEnterBlock([SayStmt('You enter.'), WinStmt('You won!')])

```
ROOM_CHAMBER_ENTER:
PRINT "You enter."
PRINT "You won!"
HALT
ROOM_CHAMBER_END:
```

Phase 5 – Optimization (optimizer.py)

Applies three passes over the TAC to improve runtime efficiency.

Pass 1 – Constant Folding

Evaluates conditions whose values are known at compile-time and replaces them with constants, allowing dead branch elimination.

Before:

```
t1 = flag(door_open) // door_open = false at compile time
if t1 goto LABEL_A
goto LABEL_B
```

After:

```
t1 = false
goto LABEL_B // LABEL_A branch eliminated
```

Pass 2 – Dead Code Elimination

Removes unreachable instructions (code after HALT) and unused temporary assignments.

Before:

```
t1 = inv_has(key)
HALT
t2 = flag(puzzle) // Unreachable
```

After:

```
t1 = inv_has(key)
HALT
```

Pass 3 – Peephole Optimization

Applies local pattern-based rewrites: removes redundant goto-to-next-label sequences, consolidates consecutive identical PRINT statements, removes dead labels.

Before:

```
goto ROOM_ENTER_1
ROOM_ENTER_1:
PRINT "hello"
PRINT "hello"
PRINT "world"
```

After:

```
ROOM_ENTER_1:
PRINT "hello"
PRINT "world"
```

Optimization Statistics Reported

- const_fold_replacements
- const_fold_branches_eliminated
- dce_unreachable_removed
- peephole_redundant_gotos
- peephole_dead_labels

Phase 6 – Execution / Game Runtime (interpreter.py)

Executes the validated AST as an interactive text-based game. Maintains all mutable state and processes player commands in a read-eval loop.

Runtime State

```
class GameRuntime:
    rooms      : dict[str, RoomNode]
    items      : dict[str, ItemNode]
    npcs       : dict[str, NPCNode]
    flags      : dict[str, bool]
    inventory   : set[str]
    room_items : dict[str, set[str]]
    current_room: str
    game_over  : bool
```

Game Loop

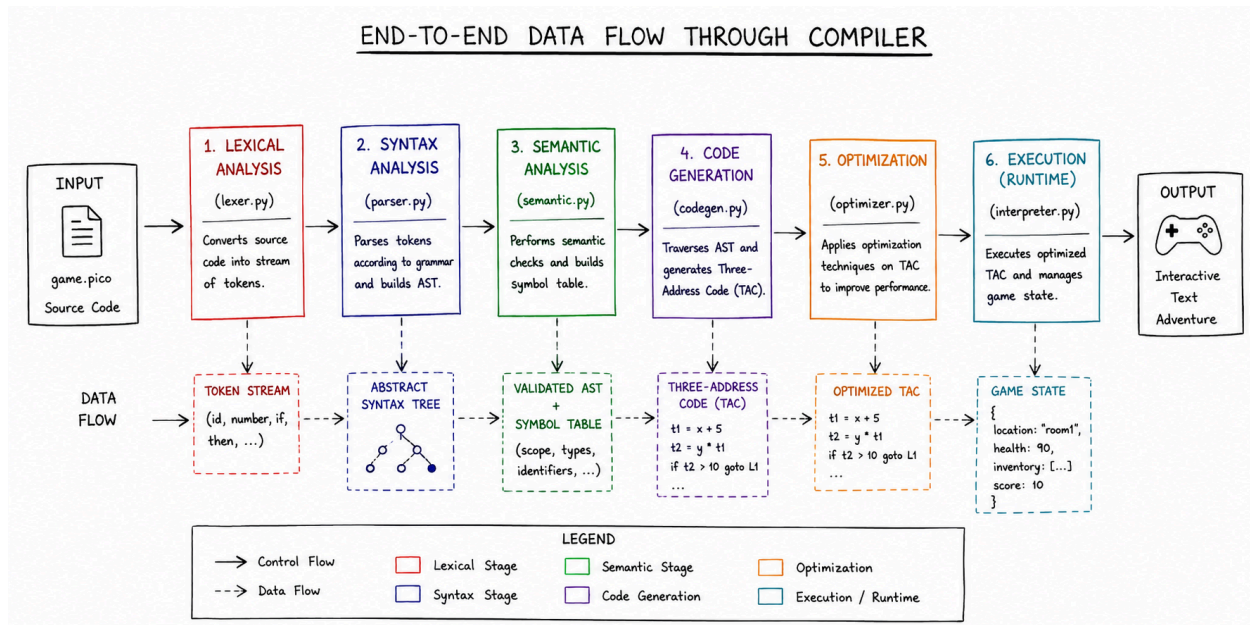
```
def run(self):
    self._enter_room(self.current_room, initial=True)
    while not self.game_over:
        raw = input('> ').strip()
        if raw.lower() in {'quit', 'exit'}: break
        self._handle_command(raw)
```

Condition Evaluation

```
def _eval_condition(self, condition):
    if isinstance(condition, ConditionFlag):
        return self.flags[condition.flag_name]
    elif isinstance(condition, ConditionInv):
        return condition.item_name in self.inventory
    return False
```

End-to-End Data Flow Example

The following traces a complete game.pico program through all six phases.



Source (game.pico)

```
world Demo:
  flag exit_unlocked = false
  player:
    inv key
  end
  item key:
    examine: "A rusty key."
  end
  room start:
    item key
    go north -> end_room requires flag exit_unlocked
  end
  room end_room:
    on enter:
      win "You escaped!"
    end
  end
  puzzle gate:
    requires inv has key
    unlock exit_unlocked
  end
end
```

Phase 4 TAC Output

```
t1 = inv_has(key)
if t1 goto PUZZLE_UNLOCKED_1
goto PUZZLE_END_1
PUZZLE_UNLOCKED_1:
exit_unlocked = true
PUZZLE_END_1:
ROOM_END_ROOM_ENTER:
PRINT "You escaped!"
HALT
ROOM_END_ROOM_END:
```

Phase 5 Optimized TAC

```
t1 = inv_has(key)
if t1 goto PUZZLE_UNLOCKED_1
PUZZLE_UNLOCKED_1:
exit_unlocked = true
ROOM_END_ROOM_ENTER:
PRINT "You escaped!"
HALT
```

Phase 6 Sample Session

```
> look
You are in start.
You see: key
Exits: north
> go north
You can't go that way yet.
> inv
Inventory: key
[Puzzle: inv has key → exit_unlocked = true]
> go north
You are in end_room.
You escaped!
```

Performance Characteristics

Phase	Time	Space	Notes
Lexer	$O(n)$	$O(t)$	n =source size, t =token count
Parser	$O(t)$	$O(a)$	a =AST node count
Semantic	$O(a)$	$O(s)$	s =symbol table size
Codegen	$O(a)$	$O(i)$	i =instruction count
Optimizer	$O(i)$	$O(i)$	Multiple passes over instructions
Runtime	$O(m)$	$O(r)$	m =player commands, r =runtime state

Debugging & Compiler Flags

Flag	Effect
--tokens	Show token stream after lexing
--ast	Show parse tree after parsing
--symbols	Show symbol table after semantic analysis
--tac	Show TAC before optimization
--optimize	Show optimization statistics
--debug	All of the above, then run the game

Multiple flags may be combined:

```
python3 main.py game.pico --tokens --symbols
python3 main.py game.pico --ast --tac
```

Error Handling

Errors are detected and reported with context at each phase. Semantic errors are accumulated and printed together so the author can fix all issues in one pass.

Phase	Example error
Lexical	Unterminated string on line 12
Syntax	Unexpected token 'end' on line 7; expected direction
Semantic	Duplicate flag declaration: 'door_open'
Semantic	Undefined room 'treasure_vault'
Semantic	Item 'torch' in inventory is undeclared

B.6 Test Suite

The test suite consists of 5 comprehensive programs in the `examples/` folder, each exercising a distinct set of language features. Together they achieve full coverage of all PicoScript constructs.

Running the Tests

Run any individual test:

```
python3 main.py examples/crypt.pico
```

Run with full debug output (tokens, AST, symbols, TAC, optimizations):

```
python3 main.py examples/crypt.pico --debug
```

Success Criteria

Result	Criteria
PASS	Compilation succeeds with no errors; all player commands produce correct output; win/lose conditions trigger at the right moment; puzzle gates unlock correctly
FAIL	Compilation error; runtime exception; output differs from expected; game state is inconsistent

Test 1 – Crypt

Objective: Verify basic room navigation, inventory-based puzzle gates, and win conditions via on-enter blocks.

Feature under test	Detail
Flag declaration	flag door_open = false
Player starting inventory	Player begins with key
Item examination	key has examine description
Item use action	gem has use action with say statement
Puzzle gate (inv)	requires inv has key → unlock door_open
Conditional exit	go north requires flag door_open
On-enter win	Entering chamber triggers win statement

Source Code

```
world Crypt:
  flag door_open = false
  player:
    inv key
  end
  item key:
    examine: "An old iron key."
  end
  room entrance:
    item gem
    go north -> chamber requires flag door_open
  end
  item gem:
    use:
      say "The gem clicks into place."
    end
  end
  room chamber:
    on enter:
      win "You escaped!"
    end
  end
  puzzle gate:
    room entrance
    requires inv has key
    unlock door_open
  end
end
```

Scenario 1A – Successful Escape

Player already has the key, so the puzzle condition is satisfied at game start. door_open is unlocked immediately and the player can walk north to trigger the win.

Player Actions	Expected Output
> look > go north > (win triggers)	You are in entrance. You see: gem Exits: north You are in chamber. You escaped!

Scenario 1B – Item Examination

Verify that examine works for items with a description, and produces no output for items without one.

Player Actions	Expected Output
> examine key > examine gem	An old iron key. (no output - gem has no examine action)

Scenario 1C – Item Use Feedback

Using the gem produces a say statement; the game continues normally.

Player Actions	Expected Output
> take gem > use gem	You take gem. The gem clicks into place.

Test 2 – Lighthouse

Objective: Verify NPC dialogue with conditional if/else/end branches and flag-based conditions.

Feature under test	Detail
NPC dialogue	npc keeper in room shore
Conditional dialogue	if flag lamp_lit ... else ... end
Flag condition	dialogue branch switches after flag changes
Item use action	oil has use action with say statement
Puzzle gate (inv)	requires inv has oil → unlock lamp_lit
Conditional exit	go north requires flag lamp_lit
On-enter win	Entering tower triggers win statement

Source Code

```
world Lighthouse:
  flag lamp_lit = false
  player:
    inv oil
  end
  item oil:
    examine: "A flask of lamp oil."
    use:
      say "You pour the oil into the lamp reservoir."
    end
  end
  room shore:
    go north -> tower requires flag lamp_lit
  end
  room tower:
    on enter:
      win "The beam sweeps out to sea. The ship is saved!"
    end
  end
  npc keeper:
    talk:
      if flag lamp_lit
        say "The lamp burns bright. Well done."
      else
        say "The lamp is dark. Find oil to light it."
      end
    end
  end
  puzzle gate:
    room shore
```

```
        requires inv has oil
        unlock lamp_lit
    end
end
```

Scenario 2A – NPC Before Puzzle is Solved

Player talks to keeper; lamp_lit is false so the else branch fires. The puzzle condition is already satisfied (player has oil), so lamp_lit unlocks immediately.

Player Actions	Expected Output
> look > talk keeper > go north	You are in shore. Exits: north The lamp is dark. Find oil to light it. You are in tower. The beam sweeps out to sea. The ship is saved!

Scenario 2B – NPC After Flag is Set

After lamp_lit becomes true the if-branch fires, demonstrating dynamic dialogue.

Player Actions	Expected Output
> talk keeper	The lamp burns bright. Well done.

Scenario 2C – Item Examination

Examine the oil flask while it is in inventory.

Player Actions	Expected Output
> examine oil	A flask of lamp oil.

Test 3 – Tomb

Objective: Verify multiple starting inventory items, lose statements that end the game, and correct selection of item use actions.

Feature under test	Detail
Multiple starting items	Player begins with torch and scarab
Lose statement	use torch → lose (game ends in failure)
Item use (say)	use scarab → say feedback
Puzzle gate (inv)	requires inv has scarab → unlock sarcophagus_open
Conditional exit	go east requires flag sarcophagus_open
On-enter win	Entering burial_chamber triggers win

Source Code

```
world Tomb:
  flag sarcophagus_open = false
  player:
    inv torch
    inv scarab
  end
  item torch:
    examine: "A flickering wooden torch."
    use:
      lose "The torch goes out. Something ancient stirs in the darkness."
    end
  end
  item scarab:
    examine: "A golden scarab amulet."
    use:
      say "The scarab glows and fits into the door."
    end
  end
  room antechamber:
    go east -> burial_chamber requires flag sarcophagus_open
  end
  room burial_chamber:
    on enter:
      win "The pharaoh sleeps peacefully. The treasure is yours!"
    end
  end
  puzzle gate:
    room antechamber
    requires inv has scarab
    unlock sarcophagus_open
  end
```

```
end
```

Scenario 3A – Wrong Choice (Lose Path)

Using the torch triggers a lose statement. The game ends in failure immediately.

Player Actions	Expected Output
<pre>> inv > examine torch > use torch</pre>	<pre>Inventory: scarab, torch A flickering wooden torch. The torch goes out. Something ancient stirs in the darkness. (game over - loss)</pre>

Scenario 3B – Correct Solution (Win Path)

Using the scarab fires a say statement. The puzzle is already satisfied so sarcophagus_open is true and the eastern exit is passable.

Player Actions	Expected Output
<pre>> examine scarab > use scarab > go east</pre>	<pre>A golden scarab amulet. The scarab glows and fits into the door. You are in burial_chamber. The pharaoh sleeps peacefully. The treasure is yours!</pre>

Test 4 – Manor

Objective: Verify flags initialised to true, on-enter say statements (not win/lose), examine-only items, and multiple flags in one world.

Feature under test	Detail
Flag initialised true	west_wing_unlocked = true at game start
Multiple flags	west_wing_unlocked and vault_cracked
Examine-only item	portrait has examine but no use action
On-enter say	west_wing on-enter fires say (not win)
Puzzle gate (inv)	requires inv has crowbar → unlock vault_cracked
Room-scoped puzzle	puzzle gate scoped to room west_wing
On-enter win	Entering vault triggers win statement

Source Code

```
world Manor:
  flag west_wing_unlocked = true
  flag vault_cracked = false
  player:
    inv crowbar
  end
  item crowbar:
    examine: "A heavy iron crowbar. Useful for prying things open."
  end
  item portrait:
    examine: "A stern portrait. Its eyes seem to follow you."
  end
  room foyer:
    item portrait
    go west -> west_wing requires flag west_wing_unlocked
  end
  room west_wing:
    on enter:
      say "A cold draft sweeps through the darkened corridor."
    end
    go south -> vault requires flag vault_cracked
  end
  room vault:
    on enter:
      win "You crack the vault. Inside: the missing will."
    end
  end
  puzzle gate:
    room west_wing
```

```

        requires inv has crowbar
        unlock vault_cracked
    end
end

```

Scenario 4A – Exploration and Examine-Only Item

Portrait has no use action. The western exit is open from game start because `west_wing_unlocked = true`.

Player Actions	Expected Output
<pre> > look > examine portrait > go west > look </pre>	<pre> You are in foyer. You see: portrait Exits: west A stern portrait. Its eyes seem to follow you. You are in west_wing. A cold draft sweeps through the darkened corridor. Exits: south </pre>

Scenario 4B – Puzzle Unlock and Win

Player has crowbar, so the room-scoped puzzle in `west_wing` unlocks `vault_cracked`. Heading south now reaches the vault and triggers the win.

Player Actions	Expected Output
<pre> > go south > (in vault) </pre>	<pre> You are in vault. You crack the vault. Inside: the missing will. </pre>

Scenario 4C – Blocked Exit Before Unlock

If `vault_cracked` were still false the southern exit would be blocked.

Player Actions	Expected Output
<pre> > go south (before unlock) </pre>	<pre> You can't go that way yet. </pre>

Test 5 – Shipwreck

Objective: Verify multi-step puzzle chains, multiple puzzle gates, NPC dialogue that updates as flags change, and multi-room sequences.

Feature under test	Detail
Multiple flags (3)	hull_patched, anchor_raised, plus NPC condition
Multiple puzzle gates	Two independent gates in different rooms
NPC with conditions	Bosun dialogue changes after hull_patched is set
NPC room placement	Bosun scoped to lower_deck
Multiple on-enter say	lower_deck and upper_deck each have say blocks
Item use (say)	rope and patch_kit both have use actions
Sequential puzzle chain	hull_patched required before upper_deck; anchor_raised before helm
On-enter win	Entering helm triggers win

Source Code

```
world Shipwreck:
    flag hull_patched = false
    flag anchor_raised = false
    player:
        inv rope
    end
    item rope:
        examine: "A length of knotted rope."
        use:
            say "You lash the rope around the capstan."
        end
    end
    item patch_kit:
        examine: "Tar and canvas for sealing hull breaches."
        use:
            say "You press the patch over the crack. It holds."
        end
    end
    room lower_deck:
        item patch_kit
        on enter:
            say "Water sloshes around your ankles."
        end
        go north -> upper_deck requires flag hull_patched
    end
    room upper_deck:
        on enter:
```

```

        say "Salt wind tears at your coat."
    end
    go north -> helm requires flag anchor_raised
end
room helm:
    on enter:
        win "You steer the battered ship into harbour. You survived."
    end
end
npc bosun:
    room lower_deck
    talk:
        if flag hull_patched
            say "Good work below. Now raise the anchor."
        else
            say "Patch the hull first or we sink where we stand."
        end
    end
end
puzzle gate:
    room lower_deck
    requires inv has patch_kit
    unlock hull_patched
end
puzzle gate:
    room upper_deck
    requires inv has rope
    unlock anchor_raised
end
end

```

Scenario 5A – Full Solution Path

Complete walkthrough from lower_deck to helm demonstrating both puzzle gates and the changing NPC dialogue.

Player Actions	Expected Output
> look > talk bosun > take patch_kit > use patch_kit > talk bosun > go north > look > go north	You are in lower_deck. Water sloshes around your ankles. You see: patch_kit Exits: north Patch the hull first or we sink where we stand. You take patch_kit. You press the patch over the crack. It holds. Good work below. Now raise the anchor.

Player Actions	Expected Output
	You are in upper_deck. Salt wind tears at your coat. Exits: north You are in helm. You steer the battered ship into harbour. You survived.

Scenario 5B – Blocked Exit Before First Puzzle

Attempting to go north before taking the patch_kit is rejected.

Player Actions	Expected Output
> go north (hull_patched = false)	You can't go that way yet.

Scenario 5C – Item Examination

Rope is in starting inventory; patch_kit must be taken before it can be examined.

Player Actions	Expected Output
> examine rope > examine patch_kit (before take) > take patch_kit > examine patch_kit	A length of knotted rope. You don't have that item. You take patch_kit. Tar and canvas for sealing hull breaches.

Feature Coverage Matrix

A tick indicates the feature is actively exercised and verified in that test program.

Feature	Crypt	Lighthouse	Tomb	Manor	Shipwreck
Flag declarations	✓	✓	✓	✓	✓
Flag initialized to true				✓	
Multiple flags				✓	✓
Player inventory	✓	✓	✓	✓	✓
Multiple starting items			✓	✓	✓
Item examination	✓	✓	✓	✓	✓
Item use action	✓	✓	✓		✓
Win statements	✓	✓	✓	✓	✓
Lose statements			✓		
Room navigation	✓	✓	✓	✓	✓
Conditional exits	✓	✓	✓	✓	✓
On-enter blocks	✓		✓	✓	✓
Say in on-enter				✓	✓
Puzzle gates	✓	✓	✓	✓	✓
Inventory conditions	✓	✓	✓	✓	✓
Flag conditions		✓		✓	✓
NPC dialogue		✓			✓
Conditional dialogue		✓			✓
NPC room placement					✓
Examine-only items				✓	
Multiple puzzle gates					✓
Multi-room sequences					✓

Debugging a Failed Test

Use the `--debug` flag for full pipeline output, or combine individual flags to isolate the failing phase:

```
python3 main.py examples/crypt.pico --debug      # All phases + game
python3 main.py examples/crypt.pico --tokens     # Lexer output only
python3 main.py examples/crypt.pico --ast        # Parse tree only
python3 main.py examples/crypt.pico --symbols    # Symbol table only
python3 main.py examples/crypt.pico --tac        # Raw TAC only
python3 main.py examples/crypt.pico --optimize   # Optimization stats
```

B.7 Team Reflection

Challenges Faced & Solutions

Challenge 1: Bridging Declarative Syntax with Procedural Compilation

PicoScript uses a declarative syntax for defining game worlds (rooms, NPCs, items, puzzle gates), but the underlying compiler pipeline needed to translate this into procedural Three-Address Code (TAC) instructions. The mismatch between "declare once" semantics and "execute sequentially" code generation required careful state management.

We implemented a two-phase semantic analysis and code generation pipeline. The semantic analyzer first collects all declarations and validates type constraints, while the code generator maintains a symbol table mapping names to runtime memory locations. This separation allowed us to:

- Validate all declarations before code generation
- Generate deterministic TAC that properly initializes game state
- Catch declaration order dependencies early

Challenge 2: Complex Control Flow in Puzzle Gates and Conditionals

The language supports conditional blocks (`if/else/end`) in dialogue trees and puzzle gates with "requires" conditions. Translating these into jump-based TAC without proper label management led to unreachable code, infinite loops, and incorrect control flow.

We built a rigorous label generation system where every branch destination is explicitly labeled (e.g., `PUZZLE_DOOR_OPEN_UNLOCK_1`). We also introduced the optimizer to eliminate dead code, dead labels, and perform constant folding on boolean conditions. This gave us confidence that generated code was structurally sound.

Challenge 3: Inventory and Flag State Consistency

The language supports typed flags and inventory tracking, but ensuring these persist correctly across room transitions, NPC interactions, and puzzle unlocks was complex. Type checking at compile-time wasn't enough; we needed runtime safety.

We implemented a strict type system in the semantic analyzer that:

- Declares all flags with explicit types at program start
- Performs type checking on all assignments and comparisons
- Prevents type mismatches at compile time
- The interpreter enforces these rules at runtime, rejecting invalid operations

Challenge 4: Defining and Documenting a Custom Language Grammar

Creating a clean, unambiguous grammar for a domain-specific language while keeping it beginner-friendly was challenging. Early grammar drafts had shift/reduce conflicts and ambiguities around optional blocks.

We iteratively refined the grammar using reference documentation and bottom-up parsing (shift-reduce parser). We documented the final grammar in `references/grammar.txt` and validated it against real example programs, ensuring consistency across all test cases.

What We Learned About Compiler Design

The Power of Separation of Concerns

We gained deep appreciation for the classic compiler pipeline architecture: **Lexer** → **Parser** → **Semantic Analysis** → **Code Generation** → **Optimization**.

Each stage had a single, clear responsibility. When bugs arose, this modular structure made debugging straightforward – we could isolate issues to specific phases and fix them in isolation.

Semantic Analysis is Critical

Early in the project, we underestimated the semantic analysis phase. We learned that catching errors early (at compile time rather than runtime) is invaluable. Semantic analysis allowed us to:

- Validate that declared items actually exist before they're referenced
- Ensure type consistency across the entire program
- Provide clear, actionable error messages to the user

Optimization Requires Multiple Passes

Our optimizer taught us that a single pass is rarely sufficient. Dead code elimination required three separate passes:

1. **Constant folding** to simplify boolean branches
2. **Unreachable code elimination** to remove code after unconditional jumps
3. **Dead label elimination** to clean up orphaned targets

This multi-pass approach is standard in real compilers, and we now understand why.

Three-Address Code is a Universal Bridge

TAC proved to be an elegant intermediate representation. It's abstract enough to hide low-level details but concrete enough to be easily optimized and interpreted. Using TAC let us:

- Generate code without worrying about specific machine architectures
- Test optimizations on human-readable code
- Run the interpreter directly on the same TAC the optimizer produced

Domain-Specific Languages Reduce Complexity

By constraining our language to text-based adventure games, we avoided many problems full-fledged compilers must solve (polymorphism, templates, memory management).

This taught us that DSLs are powerful tools: they're simpler to implement, easier to optimize, and more intuitive for their target users.

Future Improvements if We Had More Time

3.1 Advanced Optimization Passes

Currently, our optimizer performs basic dead code elimination and constant folding. We could add:

- Loop invariant code motion (once loops are supported)
- Common subexpression elimination (recognizing and caching repeated calculations)
- Strength reduction (replacing expensive operations with cheaper equivalents)
- Peephole optimization (pattern-matching small sequences of instructions)

3.2 Richer Type System

We currently support basic types (bool, string, int). Future versions could include:

- Union types (e.g., ``flag status: bool | int``)
- Struct types (e.g., ``npc character: { name: string, health: int }``)
- Arrays for managing collections of items or dialogue options
- Generic types for reusable components

3.3 Better Error Messages

Current error messages are functional but terse. We could improve developer experience with:

- Source location tracking (showing the exact line and column of errors)
- Suggested fixes (e.g., "Did you mean ``take`` instead of ``grab``?")
- Warning system for unused variables and unreachable code
- Error recovery to continue parsing and report multiple errors at once

3.4 Debugger Support

A built-in debugger would greatly help authors:

- Breakpoints on specific rooms, NPC interactions, or flag changes
- Step-through execution to trace game state changes
- Watch expressions to monitor flag and inventory changes
- Execution timeline showing the sequence of events in a playthrough

3.5 Extended Language Features

The language could expand to support:

- Loop constructs (e.g., ``repeat`` blocks for inventory iteration)
- Function definitions (reusable blocks for common patterns)
- String interpolation (embedding variables in dialogue)
- Random events (e.g., ``random choice from [option1, option2]``)
- Time-based mechanics (e.g., actions that trigger after N turns)

3.6 Compilation to Multiple Targets

Currently, we generate TAC and interpret it directly. We could compile to:

- Python bytecode for portability and performance
- JavaScript for browser-based games
- WebAssembly for standalone web apps
- C code for embedding in game engines

Individual Contributions of Each Team Member

Batool Kazmi [23K-0672]

Primary Responsibilities:

- **Lexer Design & Implementation** — Built the tokenizer (``lexer.py``) that converts raw PicoScript source into a token stream. Handled keyword recognition, string parsing, and multi-character operators.
- **Parser Development** — Implemented the shift-reduce parser (``parser.py``) that builds the Abstract Syntax Tree from tokens. Designed the grammar rules to handle nested blocks, optional clauses, and complex declarations.
- **AST Node Definitions** — Created the complete set of AST node classes (``ast_nodes.py``) representing rooms, NPCs, items, puzzle gates, and player actions.
- **Testing & Validation** — Developed and validated all six example programs (``crypt.pico``, ``inn.pico``, ``lighthouse.pico``, ``manor.pico``, ``shipwreck.pico``, ``tomb.pico``). Ensured the parser correctly handles edge cases.

Key Achievements:

- Designed a clean, intuitive grammar that feels natural for game developers
- Built a robust tokenizer that gracefully handles comments, strings, and symbols
- Created comprehensive example programs that showcase language features

Rabia Zulfiqar [23K-0851]

Primary Responsibilities:

- **Semantic Analysis** — Implemented the semantic analyzer (``semantic.py``) that validates type safety, checks symbol resolution, and ensures all references are valid.
- **Code Generation** — Built the TAC generator (``codegen.py``) that translates the AST into Three-Address Code, managing memory allocation and control flow.
- **Optimization Pipeline** — Designed and implemented the optimizer (``optimizer.py``) with multi-pass dead code elimination, constant folding, and unreachable code detection.

- Interpreter Implementation — Created the runtime interpreter (`interpreter.py`) that executes TAC instructions, managing game state, player interactions, and puzzle progression.
- Documentation — Wrote comprehensive documentation including the proposal, optimizer explanation, and architecture notes.

Key Achievements:

- Built a mathematically sound semantic analysis framework that catches errors early
- Engineered a sophisticated optimizer that improves code quality through multiple passes
- Implemented a faithful game interpreter that correctly executes all language features
- Created clear, detailed documentation explaining compiler phases and design decisions

Collaborative Efforts

As a team, both of us worked together on:

- Language design — Jointly crafted the PicoScript syntax and semantics
- Debugging — Systematically identified and fixed issues across the pipeline
- Code review — Ensured quality through peer review of all major components
- Example programs — Created diverse, interesting game worlds that test all features

Conclusion

Building PicoScript taught us that compiler design is fundamentally about clear thinking, careful separation of concerns, and rigorous testing. By breaking the problem into manageable phases (lexing, parsing, semantic analysis, code generation, optimization), we built a compiler that is both correct and maintainable. The project reinforced that domain-specific languages are powerful tools for solving specific problems elegantly, and that investing time in proper architecture pays dividends when debugging and extending the system.

We're proud of what we've accomplished and grateful for the learning opportunity this project provided!